

ЛАБОРАТОРНАЯ РАБОТА №5

Операции с файлами и каталогами

1. Цель работы

1. Копирование файлов и каталогов.
2. Перемещение/переименование файлов и каталогов.
3. Создание каталогов.
4. Удаление файлов и каталогов.
5. Создание жестких и символических ссылок.

2. Теоретическая часть

Групповые символы

Имена файлов используются в командной оболочке повсеместно, она поддерживает специальные символы, помогающие быстро определять группы имен файлов. Эти специальные символы называют групповыми символами (wildcards). Групповые символы (также известны как символы подстановки (globbing)) позволяют выбирать имена файлов по шаблону.

В таблице перечислены групповые символы и их соответствия.

Групповой символ	Соответствует
*	Любая последовательность любых символов
?	Любой один символ
[символы]	Любой один символ из указанного множества символов
[!символы]	Любой один символ, не принадлежащий указанному множеству символов
[:класс:]	Любой один символ, принадлежащий указанному классу

В следующей таблице представлены наиболее часто используемые классы символов.

Класс символов	Соответствует
[:alnum:]	Любой алфавитно-цифровой символ
[:alpha:]	Любой алфавитный символ
[:digit:]	Любой цифровой символ
[:lower:]	Любая буква в нижнем регистре
[:upper:]	Любая буква в верхнем регистре

Групповые символы позволяют конструировать сложные критерии выбора имен файлов. В следующей таблице перечислены некоторые примеры шаблонов и их соответствия.

Шаблон	Соответствует
*	Все имена файлов
g*	Все имена файлов, начинающиеся с символа «g»
b*.txt	Все имена файлов, начинающиеся с символа «b», за которым следует любое число других символов, и заканчивающиеся на «.txt»
Data???	Все имена файлов, начинающиеся с символов «Data», за которыми следуют ровно три любых символа
[abc]*	Все имена файлов, начинающиеся с символа «a», «b» или «c»
BACKUP.[0-9][0-9][0-9]	Все имена файлов, начинающиеся с символов «BACKUP.», за которыми следуют ровно три цифровых символа
[:upper:]*	Все имена файлов, начинающиеся с буквы в верхнем регистре
[:!digit:]*	Все имена файлов, не начинающиеся с цифры
*[:lower:]123]	Все имена файлов, заканчивающиеся буквой в нижнем регистре или цифрой «1», «2» или «3»

mkdir — создание каталогов

Команда `mkdir` используется для создания каталогов. Вызывается она следующим образом:

```
mkdir каталог...
```

Примечание к форме записи: Три точки в описании команды, следующие за аргументом, говорят о том, что аргументов может быть несколько; то есть в данном случае команда

```
mkdir dir1
```

создаст один каталог с именем `dir1`, а команда

```
mkdir dir1 dir2 dir3
```

создаст три каталога с именами `dir1`, `dir2` и `dir3`.

cp — копирование файлов и каталогов

Команда `cp` копирует файлы и каталоги. Ее можно использовать двумя разными способами:

```
cp item1 item2
```

чтобы скопировать один файл или каталог `item1` в файл или каталог `item2`, и

```
cp элемент... каталог
```

чтобы скопировать несколько элементов (файлов или каталогов) в указанный каталог.

В следующем таблице перечислены некоторые параметры (короткие и эквивалентные им длинные), наиболее часто используемые с командой `cp`.

Параметр	Значение
-a, --archive	Скопировать файлы и каталоги со всеми атрибутами, включая идентификаторы владельцев и права доступа. Без этого параметра копии обычно получают значения атрибутов по умолчанию, определенных для пользователя, выполняющего копирование
-i, --interactive	Запрашивать у пользователя подтверждение перед перезаписью существующего файла. Если этот параметр отсутствует, команда <code>cp</code> просто перезапишет существующие файлы
-r, --recursive	Рекурсивно копировать каталоги и их содержимое. Это обязательный параметр (или параметр <code>-a</code>) при копировании каталогов
-u, --update	При копировании файлов из одного каталога в другой копировать только файлы, отсутствующие в каталоге назначения или более новые
-v, --verbose	Выводить информационные сообщения в процессе копирования

`mv` — перемещение и переименование файлов

Команда `mv` выполняет операции перемещения и переименования файлов в зависимости от особенностей использования. В любом случае исходный файл исчезает после операции. Команда `mv` используется почти так же, как команда `cp`:

```
mv item1 item2
```

перемещает или переименовывает файл, или каталог `item1` в `item2`.

```
mv элемент... каталог
```

перемещает один или более элементов из одного каталога в другой.

Команда `mv` поддерживает множество тех же параметров, что и команда `cp`, как показано в таблице

Параметр	Значение
-i, --interactive	Запрашивать у пользователя подтверждение перед перезаписью существующего файла. Если этот параметр отсутствует, команда <code>mv</code> просто перезапишет существующие файлы
-u, --update	При перемещении файлов из одного каталога в другой перемещать только файлы, отсутствующие в каталоге назначения или более новые
-v, --verbose	Выводить информационные сообщения в процессе перемещения

rm — удаление файлов и каталогов

Команда `rm` используется для удаления (`remove`) файлов и каталогов, например:

`rm элемент...`

где элемент — это один или несколько файлов, или каталогов.

В следующей таблице перечислены некоторые параметры, наиболее часто используемые с командой `rm`.

Параметр	Значение
-i, --interactive	Запрашивать у пользователя подтверждение перед удалением существующего файла. Если этот параметр отсутствует, команда <code>rm</code> просто удалит существующие файлы
-r, --recursive	Рекурсивно удалить каталоги. То есть вместе с каталогом будут удалены все его подкаталоги. Это обязательный параметр при удалении каталогов
-f, --force	Игнорировать отсутствующие файлы и не запрашивать подтверждения. Этот параметр отменяет действие параметра <code>--interactive</code>
-v, --verbose	Выводить информационные сообщения в процессе удаления

ln — создание ссылок

Команда `ln` применяется для создания жесткой или символической ссылки.

Ее можно использовать одним из двух способов:

`ln файл ссылка`

создает жесткую ссылку.

`ln -s элемент ссылка`

создает символическую ссылку, где элементом может быть файл или каталог.

Жесткие ссылки

Жесткие ссылки — это первоначальный способ создания ссылок в Unix; символические ссылки — более позднее изобретение. По умолчанию каждый файл имеет одну жесткую ссылку, определяющую его имя. Создавая жесткую ссылку, мы создаем дополнительную запись в каталоге для файла. Жесткие ссылки имеют два важных ограничения.

- Жесткая ссылка не может указывать на файл за пределами собственной файловой системы. Это означает, что ссылка не может указывать на файл, находящийся в другом разделе диска.
- Жесткая ссылка не может указывать на каталог.

Жесткая ссылка неотличима от самого файла. В отличие от символических ссылок, при выводе списка с содержимым каталогов жесткие ссылки никак не выделяются. При удалении жесткой ссылки удаляется только сама ссылка, а файл остается на месте (то есть пространство, занимаемое файлом, не освобождается), пока не будут удалены все жесткие ссылки на файл.

Символические ссылки

Символические ссылки были придуманы с целью преодолеть ограничения жестких ссылок. Когда создается символическая ссылка, в действительности создается файл особого типа, содержащий текстовый указатель на файл или каталог. В некотором отношении они действуют подобно ярлыкам в Windows, но, конечно же, появились задолго до ярлыков Windows.

Файл, на который указывает символическая ссылка, и сама символическая ссылка почти неотличимы друг от друга. Например, если попытаться что-то записать в символическую ссылку, запись будет выполнена в файл, на который она указывает. Однако при удалении символической ссылки удаляется только символическая ссылка, но не файл. Если удалить файл до того, как будет удалена символическая ссылка, ссылка останется на месте, но будет указывать в никуда. О таких ссылках говорят, что они «битые». Во многих реализациях команда `ls` выделяет битые ссылки цветом, например, красным, чтобы обратить на них внимание.

3. Практическая часть

Пример 1: Создать каталог playground в домашнем каталоге:

```
$ cd
$ mkdir playground
```

Пример 2: Создать внутри каталога playground каталоги с именами dir1 и dir2:

```
$ cd playground
/playground$ mkdir dir1 dir2
```

Пример 3: Скопировать файл passwd из каталога /etc в текущий рабочий каталог (playground). Вывести содержимое текущего каталога:

```
/playground$ cp /etc/passwd .
/playground$ ls -l
```

Пример 4: Переименовать файл passwd на имя fun:

```
/playground$ mv passwd fun
```

Пример 5: Переместить переименованный файл (fun) в каталог dir1. Затем переместить dir1 в каталог dir2. В заключении вернуть все на места. На каждом шаге проверить их содержимое командой ls:

```
/playground$ mv fun dir1
/playground$ ls -l dir1
/playground$ mv dir1 dir2
/playground$ ls -l dir2
/playground$ ls -l dir2/dir1
/playground$ mv dir2/dir1 .
/playground$ mv dir1/fun .
/playground$ ls -l
```

Пример 6: Создать в каждом каталоге жесткие ссылки для файла fun. Проверить их содержимое командой ls с индексным узлом:

```
/playground$ ln fun fun-hard
/playground$ ln fun dir1/fun-hard
/playground$ ln fun dir2/fun-hard
/playground$ ls -li
/playground$ ls -li dir1
/playground$ ls -li dir2
```

Пример 7: Создать в каждом каталоге символические ссылки для файла fun. Проверить их содержимое командой ls с индексным узлом:

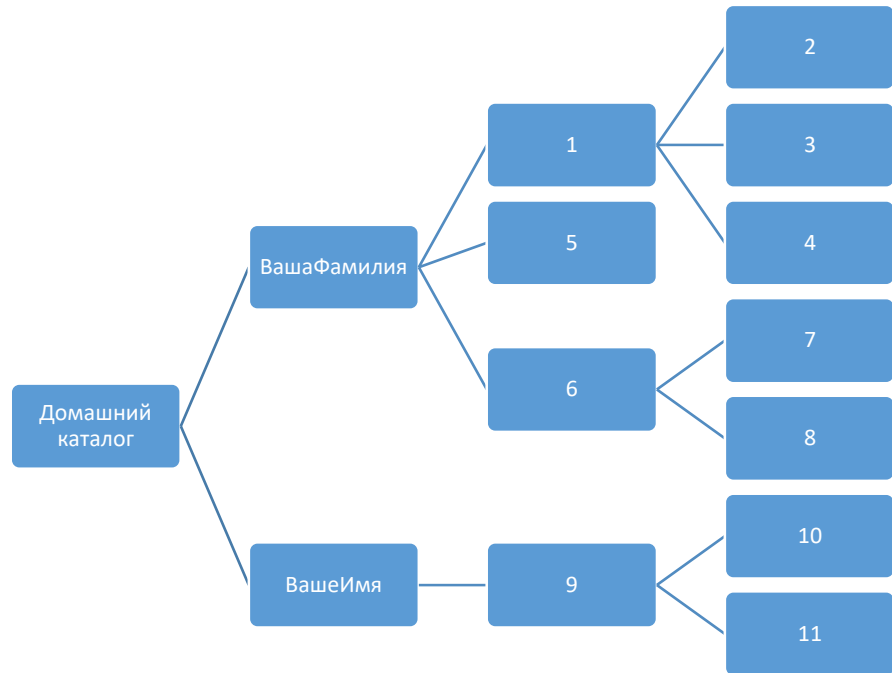
```
/playground$ ln -s fun fun-sym
/playground$ ln -s fun dir1/fun-sym
/playground$ ln -s fun dir2/fun-sym
/playground$ ls -li
/playground$ ls -li dir1
/playground$ ls -li dir2
```

Пример 8: Удалить жесткую ссылку из текущего каталога. Затем удалить файл fun в интерактивном режиме. В заключении удалить каталог playground. На каждом шаге проверить содержимое текущего каталога командой ls:

```
/playground$ rm fun-hard
/playground$ ls -l
/playground$ rm -i fun
/playground$ ls -l
/playground$ cd
$ rm -r playground
$ ls -l
```

4. Задание

1. Создать в домашнем каталоге следующую структуру подкаталогов (существующие каталоги не удалять!).



2. Скопировать файл group из каталога /etc в каталог 1 и 2. Вывести содержимое каталога 1.

3. Переименовать файл group на mygroup в каталоге 2.

4. Переместить переименованный файл (mygroup) в каталог 11. Затем переместить 11 в каталог 5. В заключении вернуть все на места. На каждом шаге проверить их содержимое командой ls.

5. Создать в каждом каталоге жесткие ссылки для файла mygroup. Проверить их содержимое командой ls с индексным узлом.

6. Создать в каждом каталоге символических ссылок для файла mygroup. Проверить их содержимое командой ls с индексным узлом.

7. Удалить жесткую ссылку из текущего каталога. Проверить его содержимое командой ls.

8. Удалить файл mygroup в интерактивном режиме. Проверить содержимое каталога командой ls.

9. Удалить все файлы, имена которых начинаются с буквы в нижнем регистре из каталога «ВашаФамилия».

10. Удалить каталоги «ВашаФамилия» и «ВашеИмя» рекурсивно.